Trabajo final
Algoritmos y estructura de datos

2do cuatrimestre 2026

Universidad Nacional Arturo Jauretche
Lemos, Gonzalo Sebastian
Flores, Jimena Ianina

Introducción:

A partir de la estructura de datos **Árbol General**, en el presente trabajo se implementaron en la clase *Estrategia.cs* diversos métodos orientados a resolver requerimientos reales dentro de una aplicación de e-commerce. En este modelo jerárquico, cada nodo representa una categoría o subcategoría capaz de organizar y contener sus respectivos elementos y productos.

Implementación y resultados:

- **Agregar(ArbolGeneral arbol, ItemCat dato, string rutaAlPadre):** Para la implementación del método Agregar, se procesó la cadena recibida (rutaAlPadre) descomponiéndola en niveles para recorrer el árbol desde la raíz. En cada nivel, el algoritmo verifica secuencialmente entre los hijos del nodo actual si la categoría ya existe; en caso contrario, crea e inserta automáticamente el nuevo nodo contenedor para garantizar la continuidad del camino. Una vez alcanzado el nodo padre final, se valida que el elemento no se encuentre previamente registrado para evitar duplicados y se procede a incorporarlo como un nuevo subárbol hijo. ***<Ir al código>***
- **Buscar(ArbolGeneral arbol, string elementoABuscar):** Para el método Buscar, se implementó una búsqueda en profundidad (DFS), la cual en la estructura de un árbol general se traduce naturalmente en un recorrido en Preorden: primero se evalúa la raíz del subárbol actual y luego se desciende de forma recursiva hacia cada uno de sus hijos. Se utiliza este enfoque porque, conceptualmente, todo árbol es un grafo dirigido y acíclico, lo que hace que DFS sea plenamente válido y equivalente a un preorden clásico. El algoritmo verifica si el nombre del elemento coincide total o parcialmente con la cadena buscada, ignorando mayúsculas y minúsculas, para añadirlo a la lista de resultados. ***<Ir al código>***
- **Todos(ArbolGeneral arbol):** Para la implementación del método Todos, se utilizó un recorrido en Preorden mediante recursión para extraer únicamente los productos del catálogo. En cada llamada, el algoritmo inspecciona el elemento almacenado en la raíz del nodo actual; si su propiedad Tipo corresponde a Producto, se añade a una lista local, ignorando los nodos que actúan como categorías o “contenedores”. Luego, se recorren secuencialmente todos los subárboles hijos de manera recursiva y se consolidan sus resultados parciales utilizando AddRange.***<Ir al código>***
- **GetURLsSEO(ArbolGeneral arbol):** Para el método GetURLsSEO, se diseñó una solución basada en un recorrido en profundidad (DFS), el cual sobre la estructura de un árbol general opera como un recorrido en Preorden: primero se

procesa y concatena la información del nodo actual y luego se desciende de manera recursiva por cada una de sus ramas. La función principal inicializa el prefijo del dominio (tienda.com) e invoca al método auxiliar DfsUrls, el cual formatea el nombre de cada categoría a formato *slug* (en minúsculas y reemplazando espacios por guiones). Para evitar redundancias en la URL, se evalúa si el nodo en curso es la raíz para no repetir el nombre de la tienda. Al alcanzar un nodo hoja (nodo.esHoja()), se determina la finalización del camino y se añade la ruta completa a la lista de resultados. ***<Ir al código>***

- **GetUrlSeoPorId(ArbolGeneral arbol, int id):** Para la implementación del método GetUrlSeoPorId, se aplicó una búsqueda en profundidad (DFS) en Preorden que combina la navegación jerárquica con la construcción dinámica de rutas. El algoritmo inicia desde la raíz con el dominio base (tienda.com) y desciende recursivamente acumulando el camino en formato amigable (*slug*), controlando no duplicar el nombre de la tienda si coincide con la raíz. En cada nodo inspecciona el identificador (Id); si coincide con el valor buscado, retorna de forma inmediata la URL construida hasta ese punto, interrumpiendo tempranamente la exploración del resto de las ramas para maximizar la eficiencia. Si el nodo actual no es el destino, el método propaga la búsqueda hacia sus subárboles hijos de manera recursiva, devolviendo la cadena correspondiente en caso de hallarlo o una cadena vacía si el elemento no existe en la estructura. ***<Ir al código>***
- **ConsultaNiveles(ArbolGeneral arbol):** Para la implementación del método ConsultaNiveles, se adoptó un recorrido por niveles, formalmente conocido como búsqueda en anchura (BFS - Breadth-First Search). Este enfoque utiliza una estructura de cola para gestionar el orden de visita de los nodos. Mientras la cola contenga elementos, se realiza un conteo inicial de su longitud para determinar la cantidad exacta de nodos que integran el nivel actual. De esta manera, el bucle procesa únicamente los nodos correspondientes a esa profundidad y encola sus respectivos hijos, evitando mezclarlos con los elementos del nivel siguiente. ***<Ir al código>***

Mejoras a futuro:

Como líneas de trabajo a futuro y disponiendo de mayor margen de tiempo, se identifican claras oportunidades de mejora tanto a nivel de arquitectura como de experiencia de usuario:

1. **Migración a plataforma Web:** Actualmente, al distribuirse como un paquete APK para Android, la solución queda restringida a dicho sistema operativo, limitando el acceso a usuarios de plataformas como Windows, macOS o iOS. Desarrollar una interfaz web responsiva permitiría centralizar la lógica, garantizar portabilidad multiplataforma y facilitar el despliegue sin depender de instalaciones locales.
2. **Optimización de la interfaz en la carga de rutas (Categorías / Slugs):** En el método de inserción actual, el usuario debe ingresar manualmente la ruta completa de categorías separada por delimitadores, lo que incrementa el

tiempo de carga y la probabilidad de cometer errores tipográficos. Una mejora sustancial radicaría en sustituir esta entrada manual por una navegación jerárquica interactiva (menús desplegables en cascada), mejorando notablemente la experiencia del cliente.

Conclusión:

El desarrollo de este proyecto nos brindó una aproximación cercana a escenarios reales del ejercicio profesional, abarcando desde la dinámica del trabajo colaborativo y el control de versiones con GitHub, hasta la realización de una solución de software orientada a la mantenibilidad y escalabilidad a futuro. Asimismo, el trabajo impulsó la resolución continua de desafíos prácticos que aportaron mejores prácticas en el sistema; por ejemplo, al optimizar la funcionalidad de búsqueda para que al ingresar un término como "*tele*", el algoritmo recupere de forma inclusiva todas las coincidencias parciales (como "*televisor*" o "*teledirigido*"), adaptándose a los patrones de uso reales donde se espera que el motor de búsqueda interprete por sí mismo la intención del usuario. En esa misma línea, se reforzó la robustez del código contemplando casos límite cotidianos, tales como el filtrado de espacios en blanco redundantes y el tratamiento insensible a mayúsculas y minúsculas.

ANEXOS:

Método agregar:

```
public void Agregar(ArbolGeneral<ItemCat> arbol, ItemCat dato, string rutaAlPadre){
            if (rutaAlPadre == null || rutaAlPadre.Trim().Length == 0){
                throw new ArgumentException("La ruta al nodo padre no puede estar vacía.", nameof(rutaAlPadre));
            }
            string[] niveles = rutaAlPadre.Split(new char[] { '/', '>' }, StringSplitOptions.RemoveEmptyEntries);
            ArbolGeneral<ItemCat> actual = arbol;
            int inicio = 0;
            if (niveles.Length > 0 && actual.getDatoRaiz() != null &&
                actual.getDatoRaiz().Nombre.Equals(niveles[0].Trim(), StringComparison.OrdinalIgnoreCase)){
                inicio = 1;
            }
            for (int i = inicio; i < niveles.Length; i++){
                string nombreNivel = niveles[i].Trim();
                ArbolGeneral<ItemCat> hijoEncontrado = null;
                foreach (var hijo in actual.getHijos()){
                    if (hijo.getDatoRaiz() != null &&
                        hijo.getDatoRaiz().Nombre.Equals(nombreNivel, StringComparison.OrdinalIgnoreCase))
                    {
                        hijoEncontrado = hijo;
                        break;
                    }
                }
                if (hijoEncontrado == null){
                    ItemCat nuevaCategoria = new ItemCat(nombreNivel, TipoElemento.Categoria);
                    hijoEncontrado = new ArbolGeneral<ItemCat>(nuevaCategoria);
                    actual.agregarHijo(hijoEncontrado);
                }
                actual = hijoEncontrado;
            }
            bool yaExisteDato = false;
            foreach (var hijo in actual.getHijos()){
                if (hijo.getDatoRaiz() != null &&
                    hijo.getDatoRaiz().Nombre.Equals(dato.Nombre, StringComparison.OrdinalIgnoreCase))
                {
                    yaExisteDato = true;
                    break;
                }
            }
            if (!yaExisteDato){
                actual.agregarHijo(new ArbolGeneral<ItemCat>(dato));
            }
        }
```

Método buscar:

```
public List<ItemCat> Buscar(ArbolGeneral<ItemCat> arbol, string elementoABuscar){
            List<ItemCat> resultados = new List<ItemCat>();
            if (arbol == null || string.IsNullOrWhiteSpace(elementoABuscar)){
                return resultados;
            }
            string texto = elementoABuscar.Trim();
            DfsBuscar(arbol, texto, resultados);
            return resultados;
        }
```

```
private void DfsBuscar(ArbolGeneral<ItemCat> nodo, string texto, List<ItemCat> resultados)
        {
            if (nodo == null) return;
            ItemCat actual = nodo.getDatoRaiz();
            if (actual != null && !string.IsNullOrEmpty(actual.Nombre))
            {
                if (actual.Nombre.Contains(texto, StringComparison.OrdinalIgnoreCase)){
                    resultados.Add(actual);
                }
            }
            foreach (var hijo in nodo.getHijos()){
                DfsBuscar(hijo, texto, resultados);
            }
        }
```

Método Todos:

```
public List<ItemCat> Todos(ArbolGeneral<ItemCat> arbol)
        {
            List<ItemCat> listaProductos = new List<ItemCat>();
            if (arbol == null) return listaProductos;
            if (arbol.getDatoRaiz().Tipo == TipoElemento.Producto){
                listaProductos.Add(arbol.getDatoRaiz());
            }
            foreach (var hijo in arbol.getHijos()){
                listaProductos.AddRange(Todos(hijo));
            }
            return listaProductos;
        }
```

Metodo GetUrlsSeo:

```
public List<string> GetURLsSEO(ArbolGeneral<ItemCat> arbol)
        {
            List<string> listaUrls = new List<string>();
            if (arbol == null){
                return listaUrls;
            }
            string dominioBase = "tienda.com";
            DfsUrls(arbol, dominioBase, listaUrls);
            return listaUrls;
        }
```

```
private void DfsUrls(ArbolGeneral<ItemCat> nodo, string rutaActual, List<string> listaUrls){
        if (nodo == null) return;
        ItemCat dato = nodo.getDatoRaiz();
        string segmento = "";
        if (dato != null && !string.IsNullOrWhiteSpace(dato.Nombre)){
            segmento = dato.Nombre.Trim().ToLower().Replace(" ", "-");
        }
        string nuevaRuta;
        if (rutaActual == "tienda.com" && (segmento.Contains("tienda") || string.IsNullOrEmpty(segmento))){
            nuevaRuta = rutaActual;
        }else{
            nuevaRuta = string.IsNullOrEmpty(segmento) ? rutaActual : $"{rutaActual}/{segmento}";
        }
        if (nodo.esHoja()){
            listaUrls.Add(nuevaRuta);
            return;
        }
        foreach (var hijo in nodo.getHijos()){
            DfsUrls(hijo, nuevaRuta, listaUrls);
        }
    }
```

Método GetUrlSeoPorId:

```
public string GetUrlSeoPorId(ArbolGeneral<ItemCat> arbol, int id)
        {
            if (arbol == null){
                return string.Empty;
            }
            string dominioBase = "tienda.com";
            string? resultado = DfsUrlPorId(arbol, dominioBase, id);
            if (resultado != null){
                return resultado;
            }
            return string.Empty;
        }
```

```
private string? DfsUrlPorId(ArbolGeneral<ItemCat> nodo, string rutaActual, int id){
            if (nodo == null){
                return null;
            }
            ItemCat dato = nodo.getDatoRaiz();
            string segmento = string.Empty;
            if (dato != null && !string.IsNullOrWhiteSpace(dato.Nombre)){
                segmento = dato.Nombre.Trim().ToLower().Replace(" ", "-");
            }
            string nuevaRuta;
            if (rutaActual == "tienda.com" && (segmento.Contains("tienda") || string.IsNullOrEmpty(segmento))){
                nuevaRuta = rutaActual;
            }else{
                nuevaRuta = string.IsNullOrEmpty(segmento) ? rutaActual : $"{rutaActual}/{segmento}";
            }
            if (dato != null && dato.Id == id){
                return nuevaRuta;
            }
            foreach (var hijo in nodo.getHijos()){
                string? resultadoHijo = DfsUrlPorId(hijo, nuevaRuta, id);
                if (resultadoHijo != null)
                {
                    return resultadoHijo;
                }
            }
            return null;
        }
```

Método ConsultaNiveles:

```
public List<List<string>> ConsultaNiveles(ArbolGeneral<ItemCat> arbol)
        {
            List<List<string>> resultado = new List<List<string>>();
            if (arbol == null) return resultado;
            Cola<ArbolGeneral<ItemCat>> cola = new Cola<ArbolGeneral<ItemCat>>();
            cola.encolar(arbol);
            while (!cola.esVacia())
            {
                int cantidadEnNivel = cola.cantidadElementos();
                List<string> elementosNivel = new List<string>();
                for (int i = 0; i < cantidadEnNivel; i++){
                    var nodoActual = cola.desencolar();
                    elementosNivel.Add(nodoActual.getDatoRaiz().Nombre);
                    foreach (var hijo in nodoActual.getHijos())
                    {
                        cola.encolar(hijo);
                    }
                }

                resultado.Add(elementosNivel);
            }
            return resultado;
        }
```